



Polymorphism (OOP's) in Java:

1. One name many forms is called as polymorphism.
2. Polymorphism is derived from 2 Greek words “poly” and “morphs”.
3. The word “**poly**” means “**many**” and “**morphs**” means “**forms**”. So, polymorphism means many forms.
4. The purpose of polymorphism is to increase the readability and understanding should be easy.
5. We can achieve polymorphism by the two ways –
 - **Overloading**
 - **Overriding**

Polymorphism by an example: - A man can have different characteristics simultaneously.

He can be a father, a son or a husband so the same man possesses different behavior in different situations.



CJC

Complete Java Classes

by Kunal Sir



Another Ex: A human with different kind of behaviors.



CJC

Complete Java Classes

by Kunal Sir

Overloading in Polymorphism:

1. Same name but different parameter is called as Overloading.
2. We can overload methods and constructor both.
3. Method overloading happens in same class as well as parent and child class.
4. Constructor overloading only happens in same class.
5. We can overload private, final and static methods.
6. Overloading is also known as “**Compile Time Polymorphism**”.
7. It is also known as “**Static Binding**”.

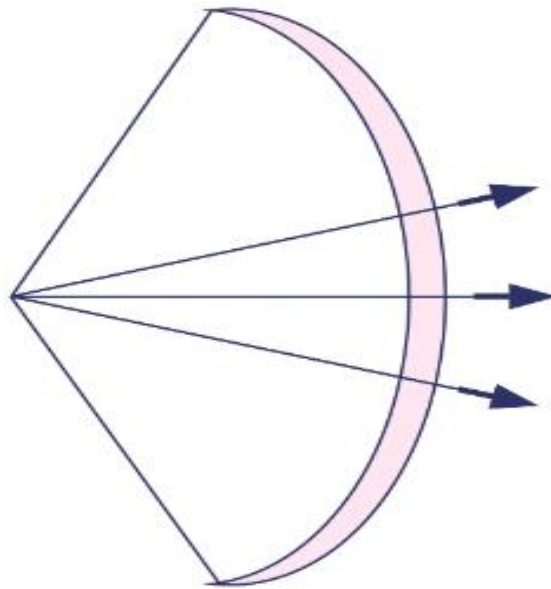


CJC

Complete Java Classes

by Kunal Sir

Overloading



Overloading Diagram

Syntax rules for the Overloading:

- Method name must be same.
- Method / Constructor parameter must be different.
- While overloading access modifier and return type does not matter.
- Overloaded methods can have different no of parameter.



- Overloaded methods can have different data type for parameter.

What is Compile Time Polymorphism?

- This type of polymorphism in java is also called static binding.
- It can be achieved by method overloading.
- In this process, an overloaded method call is resolved at compile time rather than resolving at runtime.

Understanding Program:

```
public class A
{
    public void m1()
    {
        System.out.println("m1---A");
    }
    public void m1(int x)
    {
        System.out.println("m1---int : " + x);
    }
}
```



CJC

Complete Java Classes

by Kunal Sir

```
}  
public void m1(String s1)  
{  
    System.out.println("m1---String : " + s1);  
}  
public void m1(String s1, String s2)  
{  
    System.out.println("String s1: " s1 + " String s2 : " +s2);  
}  
  
public static void main(String args[])  
{  
    A a = new A();  
    a.m1();  
    a.m1(10);  
    a.m1("CJC");  
    a.m1("Welcome", "CJC");  
}  
}
```

How Can We Overload Static Methods:

We can overload static methods according to given below



CJC

Complete Java Classes

by Kunal Sir

program-

```
public class Test
{
    public static void method_1()
    {
        System.out.println("Method_1_Test");
    }

    public static void method_1(int x)
    {
        System.out.println("Method_1_Test_Integer");
    }

    public static void main(String args[ ])
    {
        method_1();
        method_1(20);
    }
}
```

How Can We Overload Final Methods:



CJC

Complete Java Classes

by Kunal Sir

We can overload final methods according to given below program-

```
public class Test
{
    public final void method_1()
    {
        System.out.println("Method_1_Test");
    }

    public final void method_1(int x)
    {
        System.out.println("Method_1_Test_Integer");
    }

    public static void main(String args[ ])
    {
        Test t = new Test();
        t.method_1();
        t.method_1(20);
    }
}
```




How Can We Overload Private Methods:

We can overload private methods according to given below program-

```
public class Test
{
    private void method_1()
    {
        System.out.println("Method_1_Test");
    }
    private void method_1(int x)
    {
        System.out.println("Method_1_Test_Integer");
    }

    public static void main(String args[ ])
    {
        Test t1 = new Test();
        t1.method_1();
        t1.method_1(20);
    }
}
```